CS6886: Systems for Deep Learning
Experimental Report

Assignment 5: Optimising Attention on a CPU

Aravindh Sriram Kumar A G (CS26S020)

Q1. Your machine. Fill in the machine spec and compute the peak FP32 GFLOP/s (one core and all cores), the memory bandwidth and the ridge point.

Table 1: Test machine (`machine_spec.txt`). Clocks under load were measured with `perf stat`.

CPU	AMD Ryzen 7 5800X (Zen 3)	L1d cache	32 KiB per core
Cores	8 physical, 16 logical, no P/E split	L2 cache	512 KiB per core
Clock	3.8 base; 4.83 (1 core) / 4.66 (8 cores) GHz	L3 cache	32 MiB shared
SIMD	AVX2, 256-bit = 8 FP32 lanes	Memory	32 GB DDR4-3200, 1 channel
FMA units	2 per core, 4-cycle latency	Compiler	clang 14, Ubuntu 22.04

Peak compute. One vector FMA does 8 lanes × 2 FLOPs (multiply + add), and a core can start 2 FMAs per cycle, so one core does $8 \times 2 \times 2 = 32$ FLOP/cycle.

$$P_1 = 32 \times 4.834 \text{ GHz} = 154.7 \text{ GFLOP/s}, \qquad P_8 = 8 \times 32 \times 4.658 \text{ GHz} = 1192 \text{ GFLOP/s}.$$

The clock used is the one measured while all FMA units are busy, not the 3.8 GHz base clock. A small FMA benchmark reached 152.4 and 1179.3 GFLOP/s (98.5 % and 98.9 %), so the formula is right. With 16 threads it stays at about 1160: the second hardware thread on a core cannot add FLOPs because the two FMA units are already full.

Memory bandwidth. One 64-bit channel at 3200 MT/s gives 8 B × 3200 MT/s = 25.6 GB/s. A STREAM-style triad measured 16.1 GB/s with 1 thread and 15.2 GB/s with 8, so a single core already uses all the bandwidth there is.

Ridge point = peak / bandwidth (Table 2). In simple terms: a kernel that does fewer FLOPs per DRAM byte than the ridge waits on memory; a kernel above it can only be limited by the arithmetic units. With 8 cores the ridge moves about 8× to the right, because cores add compute but not bandwidth.

Table 2: Peak compute, bandwidth and ridge point.

	1 core		8 cores	
	theoretical	measured	theoretical	measured
Peak FP32 (GFLOP/s)	154.7	152.4	1192	1179.3
DRAM bandwidth (GB/s)	25.6	16.1	25.6	15.2
Ridge point (FLOP/byte)	6.0	9.5	46.6	77.6

Q2. Naive baseline. Report time, GFLOP/s and % of peak. How does time change with N and d? Where does the time go?

Table 3: `naive`, one thread. % of the measured 1-core peak (152.4 GFLOP/s). Step times come from an instrumented copy of the same code.

N	d	time (ms)	GFLOP/s	% peak	$QK^\top$ (ms)	softmax (ms)	PV (ms)
1024	64	85.07	3.16	2.1	29.0	2.9	52.8
2048	64	350.83	3.06	2.0	116.8	11.8	219.9
4096	64	1441.05	2.98	2.0	495.1	48.9	901.1
4096	128	3438.84	2.50	1.6	1082.9	48.9	2306.3

Table 3 shows that each time N doubles, the time grows about 4×. This is expected, because the work grows with N^2. When d doubles, the time grows 2.39×, a bit more than the 2× extra work. The step times in the same table show why: most of the extra time is in PV. That step reads V down a column, jumping 256 bytes (512 bytes at $d = 128$) between reads, so almost every read misses the fastest cache.

The table also shows that the two matrix products take nearly all the time and the softmax is small. GFLOP/s stays at about 3, only 2 % of peak, for every input. The reason is that the code works on one number at a time

and every step waits for the previous one. Each output is a long sum (`acc += a*b`), and each multiply–add must wait 4 cycles for the result before it. The compiler is not allowed to reorder a floating-point sum, so it cannot use the 8-wide vector units. One multiply–add every 4 cycles is 1/64 of what a core can do. Memory is not the problem: the program reads only about 0.17 GB/s from DRAM, about 1 % of the bandwidth available.

Q3. Optimisations. For each variant, report the same measurements plus speedup over `naive`. Explain what each change does and what limits performance afterwards.

Table 4: Single-threaded variants. Top: time and speedup over `naive`. Bottom: GFLOP/s and % of the measured 1-core peak (152.4 GFLOP/s).

	$N = 1024$, $d = 64$		$N = 2048$, $d = 64$		$N = 4096$, $d = 64$		$N = 4096$, $d = 128$	
Variant	ms	speedup	ms	speedup	ms	speedup	ms	speedup
naive	85.068	1.0×	350.830	1.0×	1441.053	1.0×	3438.837	1.0×
loops	11.505	7.4×	46.994	7.5×	222.167	6.5×	337.870	10.2×
tiled	5.242	16.2×	20.814	16.9×	113.522	12.7×	148.317	23.2×
fused	5.505	15.5×	21.271	16.5×	85.197	16.9×	117.638	29.2×
best	2.589	32.9×	10.345	33.9×	41.135	35.0×	77.091	44.6×
	GFLOP/s	% peak	GFLOP/s	% peak	GFLOP/s	% peak	GFLOP/s	% peak
naive	3.2	2.1	3.1	2.0	3.0	2.0	2.5	1.6
loops	23.3	15.3	22.9	15.0	19.3	12.7	25.4	16.7
tiled	51.2	33.6	51.6	33.9	37.8	24.8	57.9	38.0
fused	48.8	32.0	50.5	33.1	50.4	33.1	73.0	47.9
best	103.7	68.0	103.8	68.1	104.4	68.5	111.4	73.1

(a) `loops`: loop reordering. In `naive`, every output is a single long sum in which each step depends on the step before it, so the processor handles one number at a time. In `loops`, K is transposed once at the start and the loop order is changed so that the innermost loop updates a whole row of outputs. The elements of that row are independent of one another, so the compiler can process 8 of them with a single vector instruction. The order of additions is unchanged, so the result is identical to `naive`, bit for bit. As Table 4 shows, this variant is 6.5–10× faster than `naive`. The gain is largest at $d = 128$, because the slow column-wise reading of V is also removed. Performance is nevertheless limited to 13–17 % of peak, for three reasons. The partial sums remain in memory, so every multiply–add must also read and write them. The whole of K is read again for every row, and it is too large for the fast cache. Finally, the softmax is still computed one number at a time.

(b) `tiled`: cache and register tiling. Registers are the fastest storage in the processor. In `tiled`, the outputs are computed in small blocks of 4 rows × 16 columns, and each block is held entirely in registers until it is finished. Every value loaded from memory is therefore reused 4 to 16 times instead of once. K is also rearranged so that values used together lie next to each other in memory. Each block holds 8 independent sums, which matches the needs of the hardware: a multiply–add takes 4 cycles and there are 2 units, so $4 \times 2 = 8$ independent sums keep both units busy. Table 4 shows that `tiled` is about 2× faster than `loops`. Once the matrix products are fast, the softmax, still computed one number at a time, becomes the largest cost. At $N = 4096$ a second cost appears: the $N \times N$ score matrix occupies 64 MiB, and the operating system must prepare this memory again on every call.

(c) `fused`: fusion without the $N \times N$ matrix. The previous variants compute all the scores, then the softmax, then the output. In `fused`, all three steps are carried out for 80 rows at a time. The scores for these rows fit in a small buffer that stays in cache and is consumed immediately, so the $N \times N$ matrix is never created and temporary memory falls from 65 MiB to 2.3 MiB. Table 4 shows that this helps only at $N = 4096$, where `fused` is 1.33× faster than `tiled`. At smaller N the score matrix already fits in the 32 MiB cache, so there is nothing to save, and `fused` is slightly slower because K must be read again for each group of rows. The softmax now accounts for 60 % of the time, because it still calls the standard `expf` function once for every number.

(d) `best`: vectorised softmax. The remaining cost is the exponential, so `best` replaces `expf` with a custom function that computes 8 values at once. It is based on the identity $e^x = 2^n \cdot e^r$: the factor 2^n is built directly from the bits of the floating-point number, and e^r is approximated by a short polynomial. The accuracy is sufficient; in fact `best` is closer to an exact (FP64) answer than `naive` is. The row maximum is also computed 8 values at a time, and the division by the row sum is replaced by a single multiplication when the output is written. As a result, the softmax becomes 9× faster (50.4 to 5.6 ms). Table 4 shows that `best` is about 35× faster than `naive` and reaches 68–73 % of peak. The remaining gap is the time spent loading data into registers to feed the multiply–add units. Larger blocks would reduce this overhead, but the processor has only

16 vector registers, and larger blocks no longer fit in them.

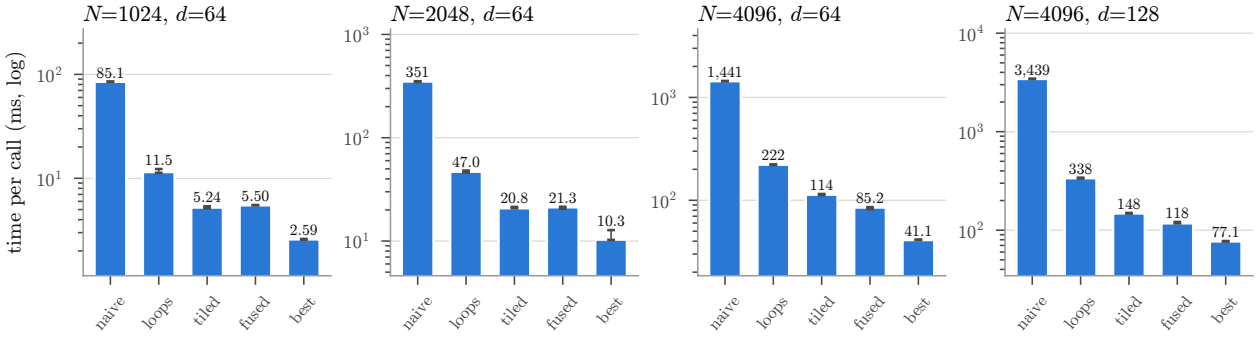


Figure 1: Time per call of the single-threaded variants on each input (log scale, labels in ms; whiskers show min–max).

Q4. OpenMP. Run the best variant with different thread counts; report time, speedup and parallel efficiency, and explain the scaling.

Each output row depends only on its own query row and the shared K and V, so rows can be computed in parallel. In `omp`, the rows are split into blocks of 20 and divided equally among the threads (`schedule(static)`), one thread per physical core. Each thread has its own score buffer, no two threads write the same row, and the result is identical to `best`.

Table 5: `omp` scaling. Speedup = time of single-thread `best` / time with T threads; efficiency = speedup/T. 16 threads means two threads per core (SMT). The best row is in bold.

	$N = 1024$, $d = 64$			$N = 2048$, $d = 64$			$N = 4096$, $d = 64$			$N = 4096$, $d = 128$		
T	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
1	2.712	0.95	95%	10.756	0.96	96%	42.508	0.97	97%	80.851	0.95	95%
2	1.379	1.88	94%	5.454	1.90	95%	21.361	1.93	96%	40.980	1.88	94%
4	0.720	3.60	90%	2.799	3.70	92%	11.021	3.73	93%	21.138	3.65	91%
8	**0.437**	**5.92**	**74%**	**1.527**	**6.77**	**85%**	**5.943**	**6.92**	**87%**	**10.942**	**7.05**	**88%**
16	0.505	5.13	32%	1.653	6.26	39%	6.109	6.73	42%	11.465	6.72	42%

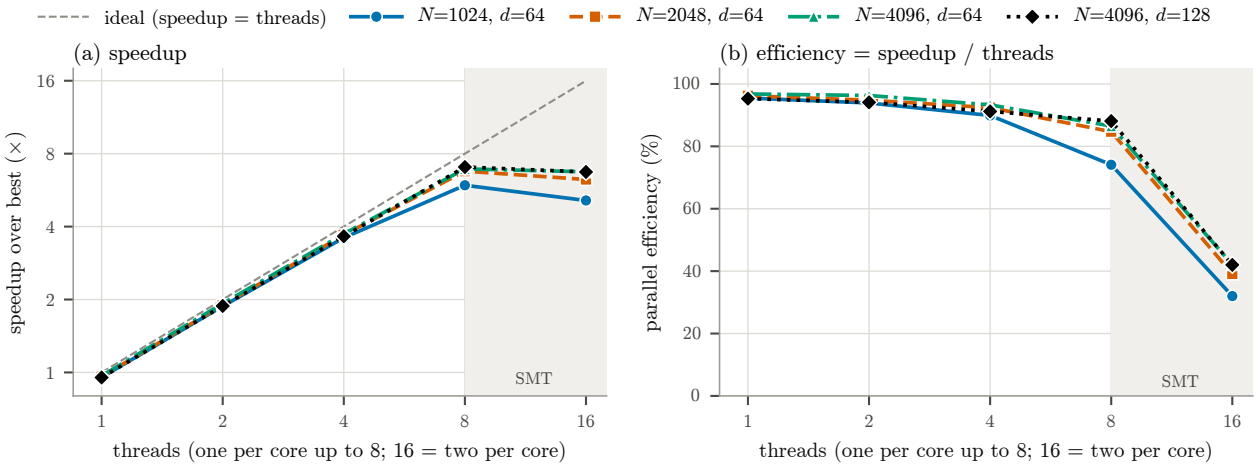


Figure 2: (a) Speedup of `omp` over single-threaded `best`, with the ideal line (grey, dashed). (b) Parallel efficiency. One line per input size; the shaded region is 16 threads (two per core).

Up to 4 threads the speedup is close to ideal (efficiency $\geq 90\,\%$), because the blocks are independent and K, V fit in the shared cache. Eight threads give the fastest time on every input (5.943 ms at $N = 4096$, $d = 64$, 6.92× over `best`), but efficiency falls to 74–88 %. OpenMP overhead is not the cause (about 0.04 ms). When all eight cores are busy, the clock drops from 4.72 to 4.54 GHz to stay within the power limit, and the cores compete for the shared cache, so each core runs about 1.17× slower. With 16 threads (two per core) the time gets slightly worse, because one thread already keeps both multiply–add units of a core busy. The smallest input scales worst (74 % at $N = 1024$): its 52 blocks do not divide evenly among 8 threads, and fixed costs are a larger share of a 0.44 ms call. A single thread of `omp` is 3–5 % slower than `best` because its smaller blocks re-read K and V more often.

Q5. Roofline. Plot the roofline with every variant at $N = 4096$, $d = 64$. Explain how the arithmetic intensity was computed and what limits each variant.

Arithmetic intensity. Arithmetic intensity (AI) is the number of FLOPs done per byte moved between DRAM and the processor. Every variant does $4N^2d = 4.295\,\text{GFLOP}$ at $N = 4096$, $d = 64$; only the bytes differ. The bytes are counted from what each algorithm must move (4 bytes per float). `naive`, `loops` and `tiled` store the score matrix S, which is $4N^2$ bytes = 64 MiB. This is larger than the 32 MiB L3 cache, so S passes through DRAM four times (written, read and written back by the softmax, read by PV), giving $16N^2 = 268\,\text{MB}$. Adding Q, K, V, O ($16Nd$) and the transposed copy of K ($8Nd$) gives about 275 MB, so AI = 4.295 GFLOP/275 MB ≈ 15.6 FLOP/byte. `fused`, `best` and `omp` never store S; only Q, K, V, O and the copy of K move, $24Nd = 6.3\,\text{MB}$, so AI = $4N^2d/24Nd = N/6 \approx 683$ FLOP/byte. DRAM reads measured with hardware counters (Table 6) agree with these estimates.

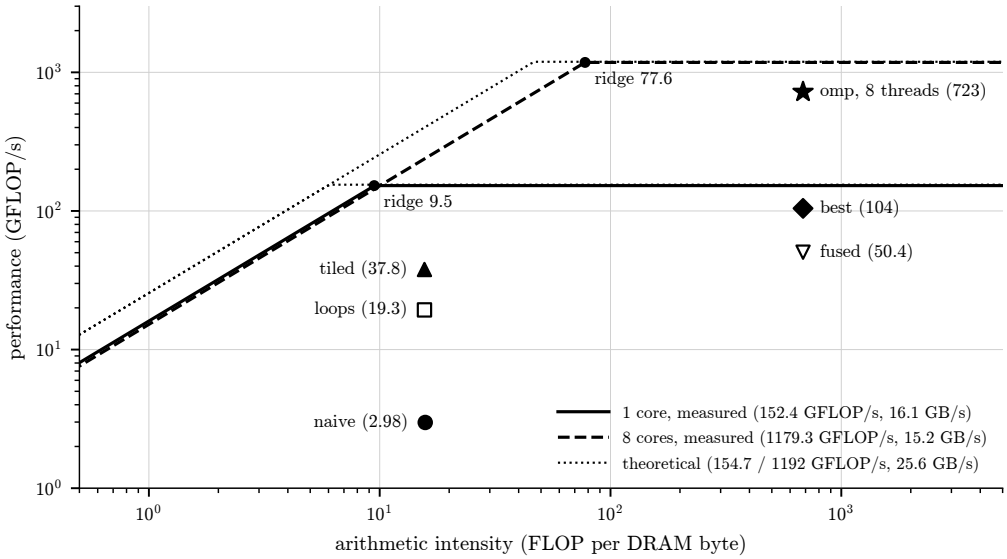


Figure 3: Roofline of the Ryzen 7 5800X with every variant at $N = 4096$, $d = 64$. Solid line: 1 core; dashed: 8 cores (both from measured peak and bandwidth); dotted: theoretical. Dots mark the ridge points; GFLOP/s in parentheses.

Table 6: Roofline points at $N = 4096$, $d = 64$. "% of roof" is relative to the measured compute roof (152.4 GFLOP/s for 1 core, 1179.3 for 8 cores).

Variant	GFLOP/s	DRAM (MB)	AI	counter reads (MB)	% of roof	Main limit
naive	2.98	272.6	15.75	238.0	2.0	one scalar multiply–add every 4 cycles
loops	19.33	274.7	15.63	233.2	12.7	partial sums kept in memory
tiled	37.83	274.7	15.63	290.8	24.8	scalar softmax, 64 MiB S set up every call
fused	50.41	6.3	682.67	2.7	33.1	scalar `expf` (60 % of time)
best	104.41	6.3	682.67	3.7	68.5	loads that feed the multiply–add units
omp (8)	722.69	6.3	682.67	6.6	61.3	as `best`, plus lower all-core clock

What limits each variant. A variant to the left of the ridge point is limited by memory bandwidth; a variant to the right is limited by computation. The ridge is at 9.5 FLOP/byte for one core and 77.6 for eight. All single-threaded variants lie to the right of the 1-core ridge, so none of them is limited by memory bandwidth, and their ceiling is the flat 152.4 GFLOP/s line.

`naive`, `loops` and `tiled` (AI ≈ 15.6) lie far below that ceiling, at 2 %, 12.7 % and 24.8 %. Their limits are inside the core, as shown in Q2 and Q3: `naive` does one scalar multiply–add every 4 cycles, `loops` reads and writes its partial sums in memory, and `tiled` spends most of its time in the scalar softmax. On 8 cores, however, these variants would be limited by memory, because $15.6 < 77.6$: at most $15.6 \times 15.2 \approx 238$ GFLOP/s, about a third of what `omp` reaches.

Fusion moves `fused`, `best` and `omp` to AI ≈ 683, far to the right of both ridges, so they are limited only by computation. `fused` reaches 33 % of the ceiling because its softmax still calls the scalar `expf`. `best` reaches 68.5 %; the rest of the time goes to the loads that bring data into registers for the multiply–add units, and to the exponential. `omp` reaches 61.3 % of the 8-core ceiling: it has the same limit as `best`, plus the lower clock and shared cache when all eight cores are busy (Q4).